



Model Predictive Control - EL2700

Linear Quadratic Regulator

Assignment 3

Division of Decision and Control Systems
School of Electrical Engineering and Computer Science
Kungliga Tekniska Högskolan

Gregorio Marchesini, Samuel Erickson Andersson, and Mikael Johansson

Background: Submission Instructions

For this assignment, we will use the solver MOSEK, which can be used together with CVXPY. If you haven't done so in the previous assignment, download an academic MOSEK license at <https://www.mosek.com/license/request/?i=acp>. You should select a personal academic license. You will then receive an email at your institutional email address with a file named `mosek.lic`. In order to find the license, the file `mosek.lic` should be placed in a folder called `mosek` in the home directory of your machine. Hence, on Linux/MAC create a folder called `mosek` and place it and save the `mosek.lic` inside of it as

```
$HOME/mosek/mosek.lic
```

On Windows, it will be the same

```
%USERPROFILE%\mosek\mosek.lic
```

Your submission should be a .ZIP file, named with your Group number (e.g., `Group1.zip`), containing:

1. a PDF document (can be a scanned document, generated from Word, Latex or others) containing the answers and motivation of the solution to the assignment. Make sure to name your document `Assignment2_STUDENT1_STUDENT2.pdf`;
2. all of the .py files with a working `task2.py` that shows your solution on the plot output.

Good Luck!

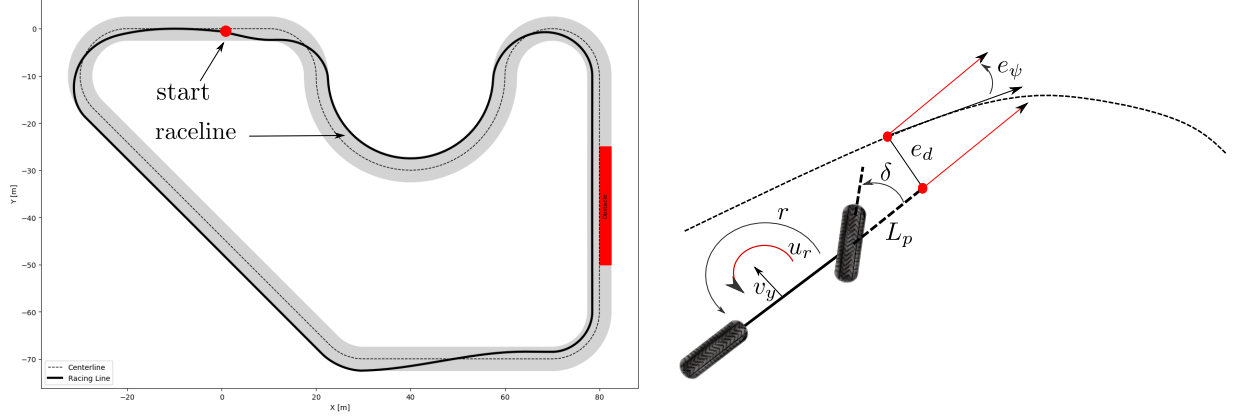


Figure 1: Racetrack circuit and optimal raceline. Red rectangle represents an obstacle to be avoided.

Introduction

In this assignment, we use a Linear Quadratic Regulator (LQR) to drive a vehicle along an optimal raceline. We first compute the raceline and a *feedforward* controller to track it, then design a feedback LQR controller to handle disturbances.

We assume a precomputed raceline (e.g., Figure 1) described by coordinates $X(s)$, $Y(s)$ and curvature $\kappa(s)$, where s is the intrinsic coordinate satisfying $0 \leq s \leq S$, with S the raceline length in meters. To track the path, we select a look-ahead point L_p meters ahead of the vehicle and define the displacement error e_d and heading error e_ψ (Figure 1, right). Assuming proximity to the raceline and constant longitudinal speed $\bar{v}$, the error dynamics follow the linear model (See Appendix if you want to understand how this model was derived)

$$\begin{bmatrix} \dot{e}_d \\ \dot{e}_\psi \\ \dot{v}_y \\ \dot{r} \\ \dot{\delta} \end{bmatrix} = \begin{bmatrix} 0 & \bar{v} & -1 & -L_p & 0 \\ 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & \frac{c_1}{m\bar{v}} & \frac{c_2}{m\bar{v}^2} - 1 & \frac{C_f}{m} \\ 0 & 0 & \frac{c_2}{I_z\bar{v}} & \frac{c_3}{\bar{v}I_z} & \frac{C_f l_f}{I_z} \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} e_d \\ e_\psi \\ v_y \\ r \\ \delta \end{bmatrix} + \begin{bmatrix} 0 \\ \bar{v} \\ 0 \\ 0 \\ 0 \end{bmatrix} \kappa + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & \frac{1}{I_z} \\ 1 & 0 \end{bmatrix} \begin{bmatrix} u_\delta \\ u_r \end{bmatrix}, \quad (1)$$

or in compact form $\dot{x}(t) = A^c x(t) + B_w^c \kappa(t) + B^c u(t)$, with state $x = [e_d, e_\psi, v_y, r, \delta]^T$, input $u = [u_\delta, u_r]^T$, and parameters given in Table 1. The states and inputs are:

Parameter	Value	Units	Parameter	Value	Units	Symbol	Description	Unit
m	1823	kg	C_f	1.6	N/rad	e_d	Displacement error	[m]
l_f	1.27	m	c_1	-3.2	N/rad	e_ψ	Heading error	[rad]
l_r	1.90	m	c_2	1.008	$N \cdot m/\text{rad}$	v_y	Lateral velocity	[m/s]
I_z	6286	$kg \cdot m^2$	c_3	-8.356	$N \cdot m/\text{rad}$	r	Yaw rate	[rad/s]
C_r	1.6	N/rad				δ	Steering angle	[rad]
						u_δ	Steering rate	[rad/s]
						u_r	Differential torque	[Nm]

Table 1: System parameters (left, center) and state/input definitions (right).

Since control actions are naturally indexed by position rather than time (a co-pilot calls “*sharp right in 100 m*”, not “*in 10 s*”) we reparametrize using the curvilinear coordinate s . At constant speed $\bar{v}$, we have $ds = \bar{v} dt$, so $\frac{d}{ds} = \frac{1}{\bar{v}} \frac{d}{dt}$, giving

$$\dot{x}(s) = \frac{1}{\bar{v}} A^c x(s) + \frac{1}{\bar{v}} B_w^c \kappa(s) + \frac{1}{\bar{v}} B^c u(s), \quad y(s) = Cx(s) = \begin{bmatrix} e_d(s) \\ e_\psi(s) \end{bmatrix}, \quad (2)$$

where y is the output error.

Design Task

In the assignment .zip file, you will find the following Python files

1. `linear_car_model.py`: Contains a class that stores the parameters of your vehicle.
2. `qp_solver.py`: This file contains an instance of the raceline optimizer we use for this assignment. You will **not** need to touch this file for this assignment.
3. `road.py`: This file contains an instance of the racetrack object that we use to store information about the racetrack. You will **not** need to touch this file for this assignment.
4. `sim.py`: This file contains the routines you will be using to simulate the performance of your LQR controller in tracking the optimal raceline computed using `qp_solver.py`.
5. `task3.py`: This is the main file that you will run to visualize the results of your simulations.

Your task will be to answer the following questions by appropriately **implementing** the code, where required, and by **motivating** your design choices. It is suggested that you use plots from your code to answer the questions!

Question 1

Inside the class `LinearCarModel` define the continuous time system matrices

$$\frac{1}{v_x} B_w^c, \frac{1}{v_x} A_w^c, \frac{1}{v_x} B^c \text{ and } C$$

as per equation (2). Remember that you can access the parameters of the vehicle using the syntax `self.parameter`. After completing this piece of code, you can directly use the method `c2d` to obtain a discrete-time system as

$$\begin{aligned} x_{k+1} &= Ax_k + Bu_k + B_w \kappa_k \\ y_k &= Cx_k. \end{aligned}$$

Question 2

Inside the file `sim.py`, we will design a *feedforward* action that tracks our reference raceline at best. In order to do so, we will use `cvxpy` to find an optimal input and state trajectory that minimizes the output error as

$$\text{minimize} \left(\sum_{k=0}^{N-1} y_k^T Q y_k + u_k^T R u_k \right) + y_N^T Q y_N \quad (3a)$$

$$\text{subject to } x_{k+1} = Ax_k + Bu_k + B_w \kappa_k \quad (3b)$$

$$|Sx_k| \leq 25 \frac{\pi}{180} \quad (3c)$$

$$y_k = Cx_k \quad (3d)$$

$$x_0 = 0_5 \quad (3e)$$

where

$$Q = \begin{bmatrix} 1000 & 0 \\ 0 & 1000 \end{bmatrix}, \quad R = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.001 \end{bmatrix} \text{ and } S = [0 \ 0 \ 0 \ 0 \ 1].$$

Note that constraint (3c) corresponds to the constraint

$$|\delta_k| \leq 25 \text{ deg}$$

limiting the maximum steering angle of the vehicle. This is indeed required for the linear model to be valid. Your task is to implement the given optimization program in (3) in the function `compute_feedforward_action` where the output should be given by two matrices representing the opti-

mal feedforward input and state sequence as

$$x^{\text{ff}} = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_N \end{bmatrix} \in \mathbb{R}^{(N+1) \times 5}, \quad u^{\text{ff}} = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \end{bmatrix} \in \mathbb{R}^{N \times 2}.$$

If you did everything correctly, your feedforward state and input trajectory will look something like Figure 4. You are welcome to play around with the parameters of the optimization (i.e., matrices Q and R), but reset the initial parameters after having fun.

Note: In Python, if you have the matrix $x^{\text{ff}} \in \mathbb{R}^{N \times n}$ then the element x_k^{ff} can be selected as the row k of x^{ff} , which is written in Python as $x^{\text{ff}}[k]$. Note that $x^{\text{ff}}[k]$ is a row vector, and, surprisingly, also $A @ x^{\text{ff}}[k]$ is a row vector that correctly represents the matrix vector multiplication (this is against the intuition that only column vectors can be multiplied with matrices, but it is important to know that this happens in Python). Thus, make sure of the fact that when you implement the dynamic constraint, all the elements in the summation are either all columns or all rows. Otherwise, some weird error will arise. The best way to make sure you have this correct is to use the attribute `.shape` for each vector in the summation to verify you have all the shapes matching.

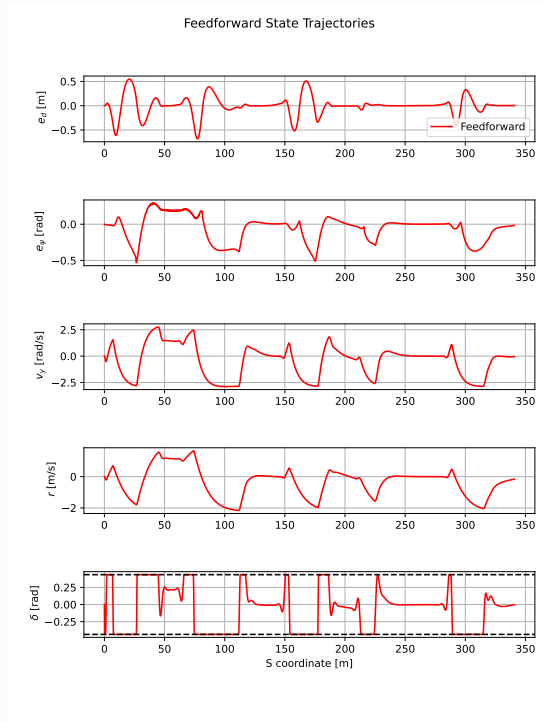


Figure 2: Reference feedforward state

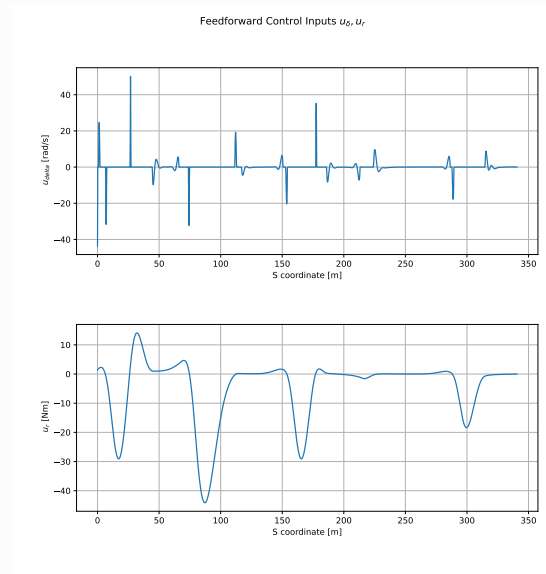


Figure 3: Feedforward reference example.

Figure 4: Feedforward example.

Question 3

Now that we have computed our feedforward action, we will design our LQR controller for the error dynamics. Namely, after defining the state and input error vectors

$$e = x_k - x_k^{\text{ff}}$$

Then the error dynamics become

$$e_{k+1} = A e_k + B(u_k - u_k^{\text{ff}})$$

which we want to stabilize using an LQR controller in the form

$$u_k = -Le_k + u_k^{\text{ff}}$$

where L is computed using the state and input matrices, $Q \succcurlyeq 0$ and $R \succ 0$ according to the Riccati equation

$$\begin{aligned} P &= Q + A^\top P A - A^\top P B (R + B^\top P B)^{-1} B^\top P A, \\ L &= (R + B^\top P B)^{-1} B^\top P A, \end{aligned}$$

and the closed-loop error dynamics become

$$e_{k+1} = (A - BL)e_k.$$

With this formulation, we want to test the performance of different controller designs on the racetrack. Namely, due to some *bumps* on the racetrack, we will be randomly offset from the raceline, and we will rely on our LQR controller to restabilize the system to its nominal state. Your task is to design an LQR controller for your system using the following cost matrices

$$\begin{aligned} Q &= \begin{bmatrix} 0.1 & 0.1 & 0.1 & 0.1 & 0.1 \end{bmatrix} & R &= \begin{bmatrix} 100 & 100 \end{bmatrix}, \\ Q &= \begin{bmatrix} 100 & 100 & 0.1 & 0.1 & 1 \end{bmatrix} & R &= \begin{bmatrix} 1 & 1 \end{bmatrix}, \\ Q &= \begin{bmatrix} 100 & 100 & 0.1 & 0.1 & 1 \end{bmatrix} & R &= \begin{bmatrix} 0.1 & 0.1 \end{bmatrix}, \end{aligned}$$

by calling the method `system.get_lqr_controller(Q, R)`. Comment on the design differences that exist between these controllers. In particular, comment on the bandwidth of the controller for the different costs (i.e., maximum and minimum of the control input) and the tracking performance (i.e., overshooting, settling time).

Question 4

As a last step, we will try to use the Bryson rule to tune our LQR controller. From the feedforward reference x^{ff} and u^{ff} , compute the maximum absolute value of each state and control variable along the trajectory. Namely, define the five coefficients^a

$$q_i = \max_{k=0, \dots, N} |x_{ki}^{\text{ff}}| \quad \forall i = 1, \dots, 5$$

and the two coefficients

$$r_i = \max_{k=0, \dots, N-1} |u_{ki}^{\text{ff}}| \quad \forall i = 1, 2.$$

With these normalizing coefficients, define the cost matrices

$$Q = \text{diag} \left(\frac{p_1}{q_1^2}, \frac{p_2}{q_2^2}, \frac{p_3}{q_3^2}, \frac{p_4}{q_4^2}, \frac{p_5}{q_5^2} \right)$$

and

$$R = \text{diag} \left(\frac{l_1}{r_1^2}, \frac{l_2}{r_2^2} \right).$$

where we use the coefficients p_i , $\forall i \in 1, \dots, 5$ and l_i , $\forall i \in 1, 2$ to tune our controller.

Try to tune your LQR controller using the tuning gains q_i and l_i .

- Comment on what happens if you increase the gains l_i compared to p_i (i.e. $l_i \gg p_i$).
- What happens if you do the opposite? (i.e. $p_i \gg l_i$)?
- Can you find a tuning that does not excessively exceed the steering limit of 35 degrees, while still preserving a good system response (low overshooting and reasonable settling time)?

^aBy the term x_{ki}^{ff} we mean the i^{th} element of the vector x_k^{ff} . Hence, spanning the index i is equivalent to spanning each single state in the state vector x_k^{ff} at step k . The same principle holds for u_{ki}^{ff} .

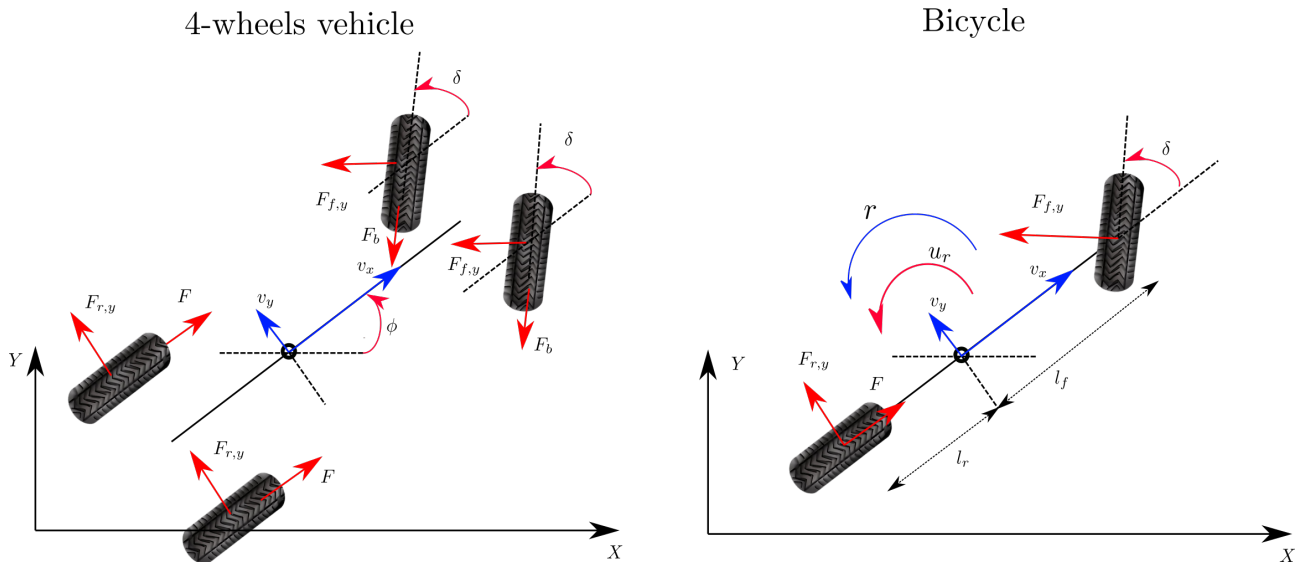


Figure 5: Dynamic Bicycle model

Appendix

Nonlinear Vehicle Model

In this section, you will find the main model applied to derive the linear dynamics that you will use in the assignment. In this assignment, we move from a kinematic model (the one used in the previous two assignments) to a dynamic model of the vehicle, which is more realistic, even if we will still make some simplifications. Namely, consider the four-wheeled vehicle schematically represented in the left panel in 5, designed in such a way that the front wheels provide braking force to decelerate and the rear wheels provide traction force to accelerate. In the left panel in Figure 5 we have depicted a simplified model of a 4-wheels vehicle where F is the driving force, $F_{r,y}$ and $F_{f,y}$ represent the force orthogonal to the rear and front tires, respectively, and lastly, F_b is the braking force. To obtain a simplified bicycle model from the 4-wheel vehicle model, we consider the following assumptions:

- We will idealize our vehicle as a bicycle model by summing forces on parallel wheels over a single wheel (left panel in Figure 5).
- Since most of the modern vehicles are able to impose differential braking on the front wheel (apply different braking force on the two front wheels), we model this effect as a control torque u_r that can be applied to favor the rotation of the bicycle model during turning.
- We introduce braking force by allowing the force F (right panel in Figure 5) to take negative values for braking.

By applying these simplifications, we arrive at the nonlinear dynamic bicycle model

$$\begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{\phi} \\ \dot{v}_x \\ \dot{v}_y \\ \dot{r} \\ \dot{\delta} \end{bmatrix} = \begin{bmatrix} v_x \cos(\phi) - v_y \sin(\phi) \\ v_x \sin(\phi) + v_y \cos(\phi) \\ r \\ \frac{1}{m}(F - F_{f,y} \sin(\delta) + m v_y r) \\ \frac{1}{m}(F_{r,y} + F_{f,y} \cos(\delta) - m v_x r) \\ \frac{1}{I_z}(F_{f,y} l_f \cos(\delta) - F_{r,y} l_r + u_r) \\ u_\delta \end{bmatrix} \quad (4)$$

where

1. v_x represents the longitudinal velocity of the vehicle in the body frame [m/s]

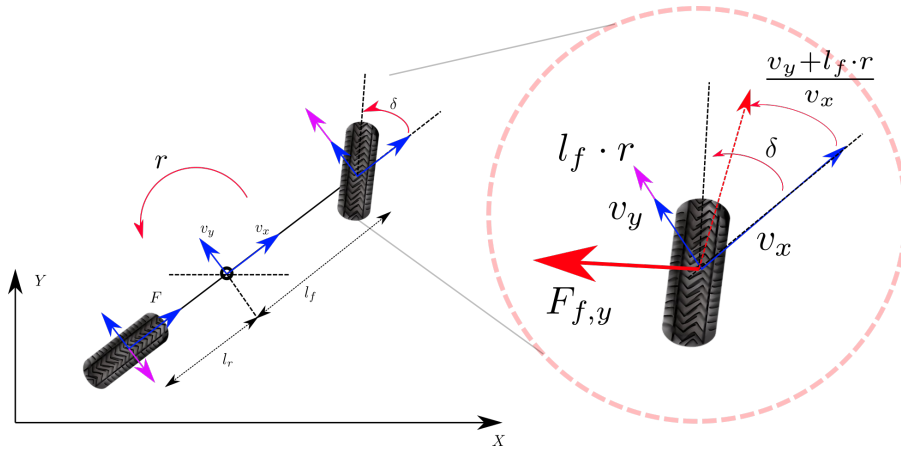


Figure 6: Force applied on the front wheel

2. v_y represents the longitudinal velocity of the vehicle in the body frame [m/s]
3. ϕ represents the heading angle [rad]
4. r is the heading rate (yaw rate) [rad/s]
5. δ is the steering angle of the wheels [rad]
6. X and Y are the inertial coordinates of the vehicle [m]

The forces imparted by the tires can be modeled for simplicity as

$$\begin{aligned} F_{f,y} &= \left(\delta - \frac{l_f r + v_y}{v_x} \right) C_f, \\ F_{r,y} &= \left(\frac{l_r r - v_y}{v_x} \right) C_r, \end{aligned} \quad (5)$$

where C_r and C_f are denoted as *cornering stiffness* coefficient and are constant parameters of tires. The additional parameter I_z appearing in the dynamics represents the moment of inertia of the vehicle around the z axis, which is pointing outward in Figure 5.

While we do not step into describing the details of the dynamics, we want to give a hint the why the forces applied by the tires are modeled as in (5), by focusing on what happens to the front wheel when the vehicle is turning. Looking at Figure 6 we see that the velocity of the front wheel is given by the velocity of the center of mass, plus a small component in the y direction due to the rotation of the vehicle (purple arrow). For the front wheel, this additional component is equal to

$$\bar{v}_y = l_f \cdot r,$$

such that the angle between the velocity vector of the front wheel and the vehicle's longitudinal axis is defined by

$$\alpha = \arctan \left(\frac{l_f \cdot r + v_y}{v_x} \right) \approx \left(\frac{l_f \cdot r + v_y}{v_x} \right),$$

where we used the small angles approximation to approximate α . This assumption is generally reasonable for moderate driving regimes where $v_x \gg v_y$. At this point, we can see that the force imparted from the front tires is proportional to the difference between the steering angle δ and the angle α as $F_{y,f} = (\delta - \alpha)C_f$, which we wrote explicitly in (5). This is in accordance with what we also experience with real vehicles. Namely, if we are going on a straight line (i.e., no lateral velocity or rotation $v_y \approx 0$ and $r \approx 0$), then a sudden change of steering produces a strong lateral force equal to the full steering angle δ (since in this case $\alpha \approx 0$). On the contrary, when we are already turning, we receive less force back from the tires for the same steering angle δ . All these arguments can be repeated on the rear tire, with the only difference that the rear tire does not have a steering angle in our model.

Linearizing the model for path following. For the linearization, we will start with the assumption that we want to track a given path with constant velocity $\bar{v}$. We will also assume that we are sufficiently aligned with the given path such that the longitudinal velocity of the vehicle v_x is approximately equal to the desired reference velocity as

$$v_x \approx \bar{v}.$$

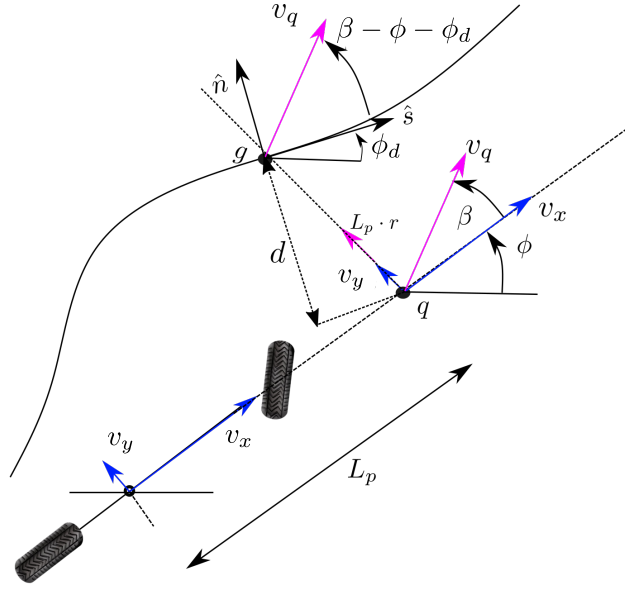


Figure 7: Path following task

Given these settings, we need to define a meaningful state error to define a successful tracking performance. To do so, we project the state of the vehicle into the local frame of the path, which we will denote F frame (or Frenet frame in the literature).

At a given goal location g along the path, the $\mathcal{F}$ frame is defined by the tangent vector $\hat{s}$ and the cross-track vector $\hat{n}$ (see top frame of reference in Figure 7). Within this frame we can define the cross-track displacement of a point, which respect to the path by the displacement d .

The successful tracking performance will revolve around setting to zero the displacement d and the heading error of the vehicle orientation with respect to the desired heading of the track. Instead of defining the displacement with respect to the center of mass of the vehicle, we will define the displacement with respect to a *preview* point in front of the vehicle named q . This is usually to allow for a smoother tracking performance.

Namely, we define the state error as

$$\begin{aligned} e_\psi &= \phi_d - \phi \\ e_d &= -d, \end{aligned}$$

and we can obtain the error dynamics by noting that

$$\begin{aligned} \dot{\phi}_d &= \kappa \bar{v} \\ \dot{d} &= v_q \sin(\beta + \phi - \phi_d). \end{aligned}$$

The velocity v_q of the point q can be approximated noting that

$$v_q = \sqrt{(v_y + L_p \cdot r)^2 + v_x^2} \approx v_x \approx \bar{v}$$

where we have used the assumption that $v_x \gg (v_y + L_p \cdot r)$ i.e. the longitudinal component of the velocity is much higher than the lateral component. Let now the angle

$$\beta = \tan\left(\frac{v_y + L_p \cdot r}{v_x}\right) \approx \frac{v_y + L_p \cdot r}{v_x} \approx \frac{v_y + L_p \cdot r}{\bar{v}},$$

Thus, we can project the velocity of the reference point q on the $\mathcal{F}$ frame to obtain the time variation of the displacement d in the $\hat{n}$ direction of the $\mathcal{F}$ frame as

$$\dot{d} = \bar{v} \cdot \sin(\beta + \phi - \phi_d)$$

which we can equivalently define based on the orientation error $e_\psi = \phi_d - \phi$ as

$$\dot{d} = \bar{v} \cdot \sin(\beta - e_\psi) = \bar{v} \sin(\beta) \cos(e_\psi) - \bar{v} \sin(e_\psi) \cos(\beta) \approx v_y + r \cdot L_p - \bar{v} e_\psi$$

where we have used the assumption that β and e_ψ are small angles for which $\cos(e_\psi) \approx 1$ and $\cos(\beta) \approx 1$. We can finally summarize the tracking error dynamics as

$$\begin{aligned} \dot{e}_\psi &= r - \kappa \bar{v} \\ \dot{e}_d &= -v_y - r \cdot L_p + \bar{v} e_\psi \end{aligned}$$

The last step we need to finalize the linear model is to write explicitly the equations of motion for v_y and r assuming a small steering angle δ such that $\cos(\delta) \approx 1$, from which we get

$$\begin{aligned}\dot{v}_y &= -\left(\frac{C_f + C_r}{m\bar{v}}\right)v_y + \left(\frac{-l_f C_f + l_r C_r - \bar{v}m}{m\bar{v}}\right)r + \frac{C_f}{m}\delta \\ \dot{r} &= \left(-\frac{C_f l_f}{\bar{v}I_z} + \frac{C_r l_r}{\bar{v}I_z}\right)v_y - \left(\frac{l_f^2 C_f}{\bar{v}I_z} + \frac{l_r^2 C_r}{\bar{v}I_z}\right)r + \frac{C_f l_f}{I_z}\delta.\end{aligned}$$

By introducing the parameters

$$c_1 = -(C_f + C_r) \quad c_2 = l_r C_r - l_f C_f \quad c_3 = -(l_f^2 C_f + l_r^2 C_r),$$

we can compactly write

$$\begin{aligned}\dot{v}_y &= \frac{c_1}{m\bar{v}}v_y + \left(\frac{c_2}{m\bar{v}^2} - 1\right)r + \frac{C_f}{\bar{v}m}\delta \\ \dot{r} &= \frac{c_2}{I_z\bar{v}}v_y + \frac{c_3}{\bar{v}I_z}r + \frac{C_f l_f}{I_z}\delta.\end{aligned}$$

and the linear dynamics of the vehicle thus become

$$\begin{bmatrix} \dot{e}_d \\ \dot{e}_\psi \\ \dot{v}_y \\ \dot{r} \\ \dot{\delta} \end{bmatrix} = \begin{bmatrix} 0 & \bar{v} & -1 & -L_p & 0 \\ 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & \frac{c_1}{m\bar{v}} & (\frac{c_2}{m\bar{v}^2} - 1) & \frac{C_f}{m} \\ 0 & 0 & \frac{c_2}{\bar{v}I_z} & \frac{c_3}{\bar{v}I_z} & \frac{C_f l_f}{I_z} \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} e_d \\ e_\psi \\ v_y \\ r \\ \delta \end{bmatrix} + \begin{bmatrix} 0 \\ \bar{v} \\ 0 \\ 0 \\ 0 \end{bmatrix} \kappa + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & \frac{1}{I_z} \\ 1 & 0 \end{bmatrix} \begin{bmatrix} u_\delta \\ u_r \end{bmatrix}.$$